GoServer: N-player Go and the Playable-Game Experiment

LeanFM project

Candidate design snapshot — 21 October 2026

Status and scope

This is a compact working book, not an accepted specification or a claim that the server is implemented. It records the current game idea so the project can resume without relying on chat history. The current canonical LeanFM requirements, protobuf schema, and implementation mapping are still drafts and do not yet express all of this design. The old book draft describes conflicting rules and is intentionally not treated as authoritative.

The immediate experiment is a playable, parameterized browser board. The longer-term goal is to use GoServer to exercise the protocol with real users. This book is built from text and an inline vector illustration; it has no conditional or external image references.

Contents

1 Game model **1**
1.1 Board and colors 1
1.2 Move, capture, and self-capture 1
1.3 Turn deadline, passes, and score 2

2 Playable board experiment **3**
2.1 Interaction 3
2.2 Computer move search 3
2.3 History and animation 3
2.4 Failure and safety invariants 4

3 Durable-work status **5**

Chapter 1

Game model

1.1 Board and colors

The board is an $N \times N$ grid. Each intersection is exactly one of Open or occupied by a player color. The player-color list is configurable (for example Black, White, Red, Blue); Wall is a special non-player color surrounding the playable grid. Wall is never a legal move or a capturable player. Its exterior side supplies liberties, so a player group touching the boundary is not treated as sealed merely because it has no on-board empty neighbor there.

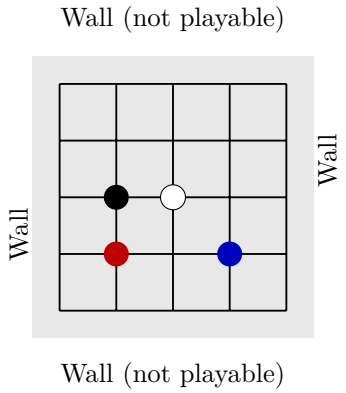


Figure 1.1: Example board state: open intersections are empty grid points; the colored stones belong to players; the shaded exterior is the non-player Wall. The Wall cannot be played or captured.

1.2 Move, capture, and self-capture

A group is a set of same-color stones connected orthogonally. A liberty is an adjacent Open intersection, with the exterior Wall boundary also supplying liberty to groups that touch the edge. A move places the current player's stone on an Open intersection. After placement, neighboring opponent groups with no liberties are removed. The removed stones are recorded as losses for their owner and returned to the capturing player's graveyard bowl. If no neighboring group is captured and the newly placed stone's group has no liberties, that own-color group is removed as a self-capture and recorded as own-color losses. Thus capture accounting is signed by color: captured other-color stones are positive; captured own-color stones are negative.

The candidate design treats a player's graveyard as a count by stone color, not as a scoring bonus. A capture changes the victim's losses; it does not award a separate capture score. The exact event

representation and ordering for moves that capture several groups must be made explicit in the accepted requirements and wire grammar.

1.3 Turn deadline, passes, and score

Every player has at most 60 seconds to produce a legal move. If the deadline expires, the turn is a Pass; the game must not wait indefinitely for a player or a search process. Pass is always a legal candidate, including in the move search. The scoring intent is

$$\text{score}(p) = \text{territory}(p) - \text{losses}(p).$$

Territory for a color is an Open region whose boundary is adjacent only to that color or Wall. Captures have no independent reward beyond the owner's losses. The terminal condition (including how many passes end a game, ties, and treatment of unowned Open regions) remains to be specified; the earlier draft's last-player-standing and Tromp–Taylor rules are not current decisions.

Chapter 2

Playable board experiment

2.1 Interaction

The browser experience should render the whole state, support human moves and computer moves, and permit forward and backward navigation through a game tree. A node represents one position; a node may have multiple child variations. The selected node is the displayed position, and every legal continuation is visibly marked as a next-move option. The view identifies the player to move, the last move, any ko-forbidden point if the chosen rules use ko, and each player's losses/graveyard counts separated by color.

The board must remain usable with keyboard navigation and accessible labels. Text on a rendered canvas uses a high-contrast color opposite the background unless the user specifies a different color. The canvas is a view of the selected game state, never an independent source of moves.

2.2 Computer move search

The experiment uses client-side Monte Carlo Tree Search (MCTS) to select a move before the same 60-second per-turn deadline. Search work must be bounded by the deadline; if no move is ready when it expires, the result is Pass. Every rollout evaluates the score of every player, rather than collapsing the game to a two-team win/loss. A natural candidate is Max^n-style vector selection: at a decision for player p, prefer the child with the best estimated component for p. This is a search-policy choice to validate, not a proven optimal strategy. It allows coalitions to emerge from incentives without hard-coding alliances. Pass remains available so search can preserve a winning position instead of filling a vital eye.

The UI must remain responsive while searching. The implementation mapping must specify cancellation, concurrency, resource bounds, seeded reproducible tests, and behavior when a worker fails. No online move oracle or remote service is currently required by this experiment.

2.3 History and animation

Users can step to a parent position, select any child variation, and step forward again without losing sibling branches. Exporting an animated GIF produces a sequence from the game root through the currently selected node; the last frame is exactly that selected state. Export does not invent moves, include later siblings, or depend on autoplay timing. The exported artifact must identify its selected endpoint and be saved within the GoServer project root.

2.4 Failure and safety invariants

Every accepted turn produces an authoritative state update; every rejected turn produces a visible error and leaves the state unchanged. A search timeout or worker failure cannot stall the game or apply a partial move: the deadline resolves to Pass when no valid move is ready. Only an authenticated player acting for the current turn may mutate a server game. Animation and export are read-only projections of recorded states, and project outputs must not escape the project root. These are candidate properties; each needs a mapped implementation control and a negative test before the feature is considered complete.

Chapter 3

Durable-work status

The current source tree contains an early Go server scaffold, but it clamps games to two players and is not a conforming implementation. The protobuf schema and typed requirements also describe only an incomplete earlier protocol. The implementation mapping is prose rather than a validated LeanFM `ImplementationSpec`. These artifacts must be reconciled with this book and with each other before source generation is authorized.

The LeanFM acceptance gates remain separate:

1. Accept the observable requirements, including game state, move and timeout behavior, scoring, game-tree navigation/export, and protocol.
2. Accept a traceable implementation specification covering the browser, MCTS execution, Go server, storage, security controls, and tests.

Only after both gates may the implementation be called complete. Each build or export should preserve its exact diagnostic and artifact path; a claimed PDF, image, or GIF is successful only after checking that the file exists and can be opened.

Sources and terminology

This is a custom n-player rules proposal, not a claim of compatibility with standard two-player Go. Published Go rules differ on matters such as self-capture, repetition, and scoring. The British Go Association's rules provide a conventional reference for terms such as groups and liberties, but they do not define this project's multi-player scoring or exterior Wall semantics. Max^n is a known multiplayer search policy; its use here is a candidate grounded in the requirement to evaluate every player's score, not a claim that one search policy is universally best.

Bibliography

[1] British Go Association, *Rules of Play*, https://www.britgo.org/files/rules/rulesofplayfull.pdf.

[2] A. Nijssen and M. Winands, "Search Policies in Multi-Player Games," *ICGA Journal*, 2013, https://journals.sagepub.com/doi/10.3233/ICG-2013-36102.